Beyond the Syllabus

Machine Learning course notes Sources: standard material, listed under References

What this file is really about:

$$\text{useful ideas that neither course covers} \rightarrow \text{one short idea each} \rightarrow \text{an interactive section to try it}$$

How to use this file. Each section is self-contained, and has an interactive section in the lecture it belongs to.

Step-up path

You need. The earlier lectures of this course.

1. Quantify uncertainty without formulas.
2. Make losses robust and intervals honest.
3. Blend models carefully.
4. Cluster and embed through graphs and kernels.
5. Separate prediction from causation.

You can stop when you can recognise when one of these tools is the right one.

Contents

Part A Maths From the Ground Up

A1 The bootstrap for confidence intervals

Resample the data with replacement many times, recompute the statistic each time, and read an interval from the percentiles. It needs no formula, so it works for medians and other awkward statistics.

A2 Fisher information

The curvature of the log-likelihood says how much data tells us about a parameter. The **Cramér-Rao bound** says no unbiased estimator has variance below $1/(nI(\theta))$. For a Bernoulli, $I(p) = 1/(p(1-p))$.

A3 The Huber loss

Squared for small errors and linear for large ones, it combines the efficiency of least squares with resistance to outliers:

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \le \delta \\ \delta(|e| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases}$$

A4 Conformal prediction

To get prediction intervals with a guarantee, compute absolute errors on a held-out calibration set, and take the $\lceil (n+1)(1-\alpha) \rceil$-th smallest as q. Then $\hat{y} \pm q$ covers a new point with probability at least $1-\alpha$, for any model, assuming only exchangeable data.

A5 Calibration of probabilities

If a model says 70%, about 70% of those cases should be positive. **Expected calibration error** bins predictions and averages the gap between confidence and accuracy, weighted by bin size.

A6 Stacking

Train several base models, and use their out-of-fold predictions as inputs to a final model. The out-of-fold step matters, because otherwise the final model learns to trust predictions made on data they memorised.

A7 Spectral clustering

Connect similar points in a graph, and compute the eigenvectors of its Laplacian $L = D - W$. Points that are connected end up close in that embedding, so K-means can then separate shapes such as rings or moons.

A8 Kernel PCA

Perform PCA on the kernel matrix instead of the covariance. Centre it first, and take the top eigenvectors. Curved structure can become linear in the implicit feature space.

A9 Local outlier factor

A point is unusual if its neighbourhood is much sparser than that of its neighbours. Because it is a ratio of local densities, it works when clusters have different densities.

A10 Graph neural networks

Each node updates its vector from those of its neighbours:

$$h'_v = \sigma\Big(W \sum_{u \in N(v) \cup \{v\}} \frac{h_u}{|N(v)| + 1}\Big).$$

After k rounds a node has information from k hops away.

A11 Causal questions

Prediction asks what will happen, and causation asks what would change if we acted. A **confounder** influences both treatment and outcome, and can even reverse a comparison. Randomisation removes this, and adjusting for measured confounders can help when randomisation is impossible.

A12 Fisher information and the Cramér-Rao bound

The **score** is $s(\theta) = \partial_\theta \log p(x \mid \theta)$, with mean zero. Fisher information is its variance:

$$I(\theta) = \mathbb{E}[s^2] = -\mathbb{E}[\partial_\theta^2 \log p].$$

Any unbiased estimator satisfies $\operatorname{Var}(\hat\theta) \geq 1/(nI(\theta))$. For a Bernoulli, $I = 1/(\theta(1-\theta))$, so the bound is $\theta(1-\theta)/n$, which the sample mean attains exactly.

A13 Why conformal prediction covers

Let $s_1, \dots, s_n$ be calibration residuals and s_{n+1} a new one, all exchangeable. Then the new score is equally likely to fall at any rank. Let $\hat q$ be the $\lceil (n+1)(1-\alpha) \rceil$-th smallest of the n scores. The chance that $s_{n+1} \leq \hat q$ is at least $1 - \alpha$. That needs no assumptions about the model or the distribution, only exchangeability.

A14 Measuring and fixing calibration

Expected calibration error bins predictions and compares confidence with accuracy:

$$\mathrm{ECE} = \sum_{b} \frac{n_b}{n} \left| \mathrm{acc}(b) - \mathrm{conf}(b) \right|.$$

Platt scaling fits $\sigma(a\,z + b)$ on a held-out set. **Temperature scaling** fits a single T in z/T, which leaves the ranking and the accuracy unchanged. **Isotonic regression** fits a non-decreasing step function, which needs more data.

A15 Spectral clustering as a relaxation

The graph Laplacian $L = D - A$ satisfies $x^\top L x = \frac{1}{2}\sum_{ij} A_{ij}(x_i - x_j)^2$. For an indicator vector, this counts the edges cut. Minimising a cut over discrete indicators is hard, so we relax to real-valued vectors and take the eigenvectors of L with the smallest eigenvalues, then run K-means on them.

Part B In Code

B1 A bootstrap interval

```
stats = [np.median(rng.choice(x, len(x))) for _ in range(2000)]
low, high = np.percentile(stats, [2.5, 97.5])
```

B2 Conformal intervals

```
scores = np.sort(np.abs(y_cal - model.predict(X_cal)))
q = scores[int(np.ceil((len(scores) + 1) * (1 - alpha))) - 1]
```

B3 Temperature scaling and calibration error

```
import numpy as np
from scipy.optimize import minimize_scalar

def ece(p, y, bins=10):
    edges = np.linspace(0, 1, bins + 1); total = 0
    for a, b in zip(edges[:-1], edges[1:]):
        m = (p > a) & (p <= b)
        if m.any():
            total += m.mean() * abs(y[m].mean() - p[m].mean())
    return total

rng = np.random.default_rng(0)
z_true = rng.normal(0, 2, 5000)
y = (rng.random(5000) < 1 / (1 + np.exp(-z_true))).astype(float)
z = 3 * z_true                                          # an overconfident model

def nll(T):
    p = np.clip(1 / (1 + np.exp(-z / T)), 1e-9, 1 - 1e-9)
    return -(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()

T = minimize_scalar(nll, bounds=(0.1, 10), method='bounded').x
print('T', round(T, 2), 'ECE before', round(ece(1 / (1 + np.exp(-z)), y), 3), 'after', rou
assert abs(T - 3) < 0.3
```

B4 Stacking without leakage

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(2000, 20, n_informative=8, random_state=0)
Xa, Xb, ya, yb = train_test_split(X, y, random_state=0)
base = [RandomForestClassifier(100, random_state=0), DecisionTreeClassifier(max_depth=5, ra

# Out-of-fold predictions, so that the meta-model never sees a base model's training fit.
meta_a = np.c_[[cross_val_predict(m, Xa, ya, cv=5, method='predict_proba')[:, 1] for m in b
meta_b = np.c_[[m.fit(Xa, ya).predict_proba(Xb)[:, 1] for m in base]].T
meta = LogisticRegression().fit(meta_a, ya)
print('stacked accuracy', round(meta.score(meta_b, yb), 3))
```

B5 Spectral clustering from scratch

```
import numpy as np
from sklearn.cluster import KMeans

t = np.linspace(0, 2 * np.pi, 100, endpoint=False)
X = np.r_[np.c_[np.cos(t), np.sin(t)], 4 * np.c_[np.cos(t), np.sin(t)]]
truth = np.r_[np.zeros(100), np.ones(100)]

A = np.exp(-((X[:, None] - X[None]) ** 2).sum(-1) / (2 * 0.3 ** 2))
np.fill_diagonal(A, 0)
d = A.sum(1)
L = np.eye(200) - A / np.sqrt(d[:, None] * d[None])     # normalised Laplacian
_, vectors = np.linalg.eigh(L)
U = vectors[:, :2]
U = U / np.linalg.norm(U, axis=1, keepdims=True)
labels = KMeans(2, n_init=10, random_state=0).fit_predict(U)
acc = max((labels == truth).mean(), (labels != truth).mean())
print('accuracy', acc)
assert acc == 1
```

Part C Practice

P1. $\delta = 1$ and $e = 3$. Find the Huber loss.

$1 \times (3 - 0.5) = 2.5.$

P2. With $n = 99$ calibration points and $\alpha = 0.1$, which ranked error is used?

The $\lceil 100 \times 0.9 \rceil = 90$th smallest.

P3. Fisher information of a Bernoulli at $p = 0.5$?

$1/0.25 = 4.$

P4. How far can information travel in a graph network after 3 rounds?

Three hops.

P5. Why must stacking use out-of-fold predictions?

In-sample predictions are overconfident, so the final model would over-trust the base models.

P6. What does a confounder do?

It influences both treatment and outcome, creating an association that is not causal.

P7. What is the Fisher information of a Bernoulli at $\theta = 0.5$?

$1/(0.25) = 4.$

P8. Does temperature scaling change accuracy?

No. Dividing logits by a positive constant does not change which is largest.